Subarray Sums Divisible by k

A *subarray* is a contiguous subset of an array. For example, `[2,3]` is a subarray of `[1,2,3,4]`, but `[2,4]` is not.

You are given an integer n, an array `a` of length n and an integer k. Compute the number of subarrays of `a` whose sum is divisible by k.

Example Test Cases:

Test Case 1
Input: $n = 6$, `a` = `[4,5,0,-2,-3,1]`, $k = 5$
Output: 7
Explanation:
There are 7 subarrays whose sums are divisible by 5: `[4,5,0,-2,-3,1]`, `[5]`, `[5,0]`, `[5,0,-2,-3]`, `[0]`, `[0,-2,-3]`, and `[-2,-3]`.

Test Case 2
Input: $n = 4$, `a` = `[2,-6,2,6]`, $k = 4$
Output: 4
Explanation:
There are 4 subarrays whose sums are divisible by 4: `[2,-6]`, `[2,-6,2,6]`, `[-6,2]`, and `[2,6]`.

Test Case 3
Input: $n = 1$, `a` = `[5]`, $k = 9$
Output: 0
Explanation: The only subarray of `5` is itself. Its sum, 5, is not divisible by 9.

Expected Time Complexity: $O(n + k)$

Input Format:
An integer n, an array `a` of length n, and an integer k.

Output Format:
Return a single integer, the number of subarrays whose sums are divisible by k.

Constraints:

1. $1 \le n \le 10^5$
2. All elements of `a` are in $[-10^4, 10^4]$.
3. $2 \le k \le 10^4$

Instructions:

You have been provided with two template files, containing the `main` and `solve` functions respectively. The `main` function, which has been completed for you, parses the input for each test case. For each test case, it calls the `solve` function, which must compute the number of subarrays whose sums are divisible by k.

The `solve` function must compute this in $O(n + k)$ time.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)

Library Usage

For this lab, you are permitted to use any modules which are part of the standard library of either `C++` or `python`. For `C++`, do not add any extra `#include` directives; the provided `bits/stdc++.h` covers all allowed modules.